VERIQTA

Linux Black Book



20 Pages of Production Linux Engineering, Kernel Behavior, Failure Analysis, and Incident Recovery

- Systemd & Services
- Processes & Scheduling
- Memory, OOM & cgroups
- Storage & Filesystems
- Linux Networking
- Security & Permissions
- Containers & Isolation
- Observability & Troubleshooting
- Real Incidents & Recovery
- Performance & Tuning

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

The Server Is Healthy, but the Application Is Dying

Production Incident: Monitoring reports healthy CPU and memory utilization, but application requests are timing out.

Key Learnings

1. Why load average, CPU utilization, and application latency can disagree.
2. Understand runnable tasks, uninterruptible sleep, and scheduler pressure.
3. Investigate /proc/loadavg, vmstat, pidstat, and pressure stall information.
4. Distinguish CPU contention from blocked I/O and dependency waits.
5. Identify whether the bottleneck exists inside the process, kernel, storage, or network.

Why Metrics Can Be Misleading

CPU Utilization (%)	Load Average (runnable + waiting)	Application Latency (user impact)
Looks Normal (e.g., 35%)	Can Be High (e.g., 8.20)	Requests Timing Out (e.g., > 10s)

Linux Task States (What the Kernel Sees)

Runnable (R)	Uninterruptible Sleep (D)	Interruptible Sleep (S)
Ready to use CPU (waiting to be scheduled)	Waiting on I/O (e.g., disk, network, NFS, database)	Waiting on an event (e.g., timer, socket, lock)

Investigation Commands

Command	What It Tells You
cat /proc/loadavg	Load average and runnable tasks
vmstat 1	CPU, memory, I/O wait, and run queue
pidstat -u -r-d 1	Per-process CPU, memory and disk I/O
cat /proc/pressure/*	CPU, memory, and I/O pressure (stall time)

Common Bottlenecks and How to Identify Them

- **CPU Contention** → High run queue, high context switches, low idle CPU
- **Blocked I/O** → High I/O wait, many tasks in "D" state, slow disk or storage
- **Network Delay** → Request timeouts, high retransmissions, slow dependency response
- **Dependency Wait** → Tasks waiting on database, API, or external service

Senior Engineer Decision: Never increase CPU capacity simply because load average is high. Establish what tasks are waiting for.

VERIQTA

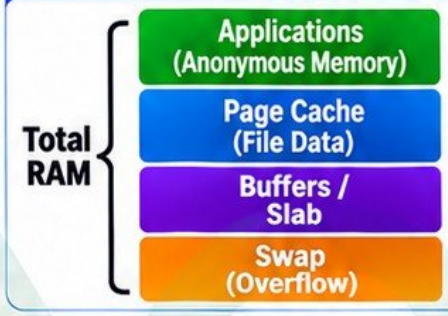
Linux Has Free Memory. Why Is the OOM Killer Active?

Production Incident: A containerized application is killed even though the host still reports available memory.

Key Learnings

1. Examine anonymous memory, page cache, reclaim, swap, and memory pressure.
2. Understand cgroup memory limits versus host-wide memory availability.
3. Investigate memory.events, memory.current, /proc/meminfo, and kernel logs.
4. Distinguish global OOM from memory-cgroup OOM.
5. Analyze memory leaks, page-cache pressure, and allocation failures.

How Linux Uses Memory



- **Anonymous Memory:** App memory (heaps, stacks, runtime).
- **Page Cache:** Cached file data for faster access (reclaimable).
- **Buffers/Slab:** Kernel metadata and object caches.
- **Swap:** Extra space when RAM is exhausted (slower).

cgroup vs Host Memory

Host Memory (System-Wide)



- Total RAM on the machine
- Shared by all processes/containers
- May still show "free" memory

cgroup Memory (Container Limit)



- A fixed limit per container
- Enforced by the kernel
- Can trigger OOM even if host has free memory

Useful Investigation Commands

Command	What It Shows
cat /proc/meminfo	Host memory breakdown (total, free, cache, swap)
cat memory.current	Current memory used by the container (cgroup)
cat memory.events	OOM events (oom, oom_kill)
dmesg -T | grep -i oom	Kernel OOM kill messages
journalctl -k | grep -i oom	OOM details and killed process

Global OOM vs cgroup OOM

Global OOM (Host Level)



- Triggered when the entire host runs out of memory.
- Kernel OOM killer selects and kills a process.
- Affects any process on the host.

cgroup OOM (Container Level)



- Triggered when a container exceeds its memory limit.
- The kernel kills processes inside that container only.
- Other containers may remain healthy.



Senior Engineer Decision: Determine which memory boundary failed before changing limits or adding RAM.

VERIQTA

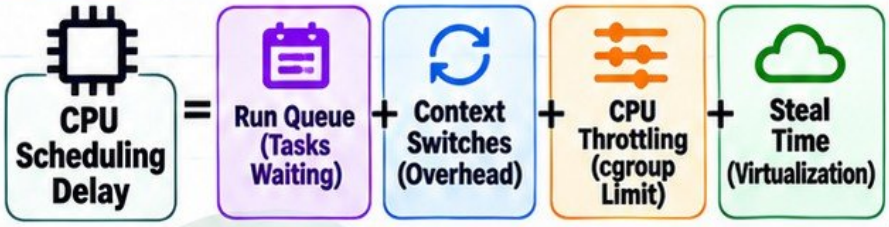
CPU Usage Is Low, but Linux Cannot Schedule Work Fast Enough

Production Incident: API latency increases sharply while CPU utilization remains below 50%.

Key Learnings

1. Examine CFS scheduling, runnable queues, context switches, and CPU affinity.
2. Investigate CPU throttling under cgroup quotas.
3. Understand steal time in virtualized environments.
4. Compare CPU utilization with CPU pressure and scheduling delay.
5. Diagnose using pidstat -w, mpstat, /proc/pressure/cpu, and cgroup CPU statistics.

What Can Make Scheduling Slow?



Key Concepts and What to Look For

CFS (Completely Fair Scheduler)	Shares CPU fairly among tasks using time slices.
Runnable Queue (runnable)	Tasks ready to run but waiting for CPU (high value = contention).
Context Switch	CPU time spent switching between tasks (more switches = more overhead).
CPU Throttling (cgroup)	Limits CPU time for a container or process (e.g., CPU quota/period).
Steal Time	CPU time taken by the hypervisor for other virtual machines.
CPU Pressure	Shows how often tasks were delayed waiting for CPU (PSI).

Investigation Commands

Command	What It Shows
pidstat -w 1	Per-process context switches and CPU wait.
mpstat -P ALL 1	CPU usage per core, including steal time (st).
cat /proc/pressure/cpu	CPU pressure (some, full) and scheduling delays.
cat /sys/fs/cgroup/cpu.stat	cgroup CPU usage, quota, throttled_time, and throttled_count.
top / htop	Run queue (load), CPU usage, and process state.

CPU Utilization vs CPU Pressure

CPU Utilization

- How much CPU time is used (e.g., 35%).
- Can look normal even when tasks are waiting.
- Does not show scheduling delays.

CPU Pressure (PSI)

- Shows time tasks were stalled waiting for CPU.
- "some" = some tasks delayed.
- "full" = all tasks delayed (for that duration).
- A better signal for latency and contention.

Architecture Trade-Off: CPU limits improve resource isolation but can introduce latency under bursty workloads.

The Disk Has Space, but Applications Cannot Write

Production Incident: A production service reports "No space left on device," but df -h shows free capacity.

Key Learnings

1. Investigate inode exhaustion using df -i.
2. Understand deleted-but-open files and why storage remains allocated.
3. Examine filesystem quotas, reserved blocks, and read-only remounts.
4. Distinguish capacity exhaustion from filesystem metadata failures.
5. Use lsof +L1, findmnt, stat, and kernel logs to narrow the cause.

Common Root Causes

- **Inode Exhaustion** (no free inodes)
- **Deleted-but-Open Files** (space still in use)
- **Filesystem Quotas** (user or group limit)
- **Reserved Blocks** (space held for root)
- **Read-Only Remount** (filesystem error or protection)

Useful Investigation Commands

Command	What It Tells You
df -h	Overall disk capacity and usage (may still show free space).
df -i	Inode usage (detect inode exhaustion).
lsof +L1	Files that were deleted but are still open by a process.
findmnt	Filesystem details, mount options, and read-only state.
stat <path>	File metadata, size, and inode information.
dmesg | grep -i (ext4|xfs|nfs)	Filesystem errors, remounts, and I/O issues in kernel logs.

Capacity Exhaustion vs Metadata Failure

Capacity Exhaustion (Actual Space)

- Disk space is truly used up (100% or near 100%).
- Large files, logs, or data growth.
- df -h shows high usage.

Metadata Failure (Inodes / Filesystem)

- No free inodes (even with free disk space).
- Filesystem errors or corruption.
- Filesystem remounted read-only (ro).

Example: Why df -h Can Be Misleading

Check	Result	Meaning
df -h	60G free	Disk space is available.
df -i	100% used	No free inodes (cannot create new files).
lsof +L1	Large deleted file	Space is still held by a running process.
findmnt	,..., (ro)	Filesystem is read-only (writes will fail).

Senior Engineer Decision: Identify the exhausted resource before deleting files or expanding storage.

The Process Is Running. The Service Is Still Dead.

Production Incident: systemctl status reports an active service, but the application cannot serve requests.

Key Learnings

1. Distinguish process existence from application readiness.
2. Examine systemd service types, notification readiness, watchdogs, and restart behavior.
3. Investigate hung threads, blocked sockets, and dependency failures.
4. Compare systemd health with application-level health checks.
5. Use systemctl show, journalctl, ss, and thread-state inspection.

Process vs Service vs Application

Process (Running)
A process exists and has a PID.

→

Service (systemd)
systemd manages the process (lifecycle, restart, dependencies).

→

Application (Ready)
The service is actually healthy and serving requests.

Important systemd Concepts

Concept	Meaning
Service Types	simple, forking, oneshot, notify (affects readiness).
Notification Readiness	The app tells systemd it is ready (Type=notify).
Watchdog	systemd restarts the service if it stops responding.
Restart Behavior	Restart=, RestartSec= (automatic recovery policy).
Dependencies	Requires=, After= (controls startup order).

Useful Investigation Commands

Command	What It Tells You
systemctl status <svc>	Service state and recent events.
systemctl show <svc>	Service configuration (Type, Restart, Watchdog, deps).
journalctl -u <svc> -f	Real-time logs for the service.
ss -tulpn | grep :<port>	Whether the app is actually listening on the expected port.
ps -eLf | grep <app>	Threads and their states (e.g., D, R, S).

Common Failure Scenarios

Hung Threads	Process is running but threads are blocked (e.g., I/O, lock).
Blocked Sockets	Cannot accept new connections (e.g., backlog full, port not listening).
Dependency Failure	Service is up but a required dependency (e.g., DB, queue) is unreachable.
Restart Loop	Service keeps restarting due to persistent failure.

Production Insight: Restart policies detect process termination. They do not automatically detect every application deadlock.

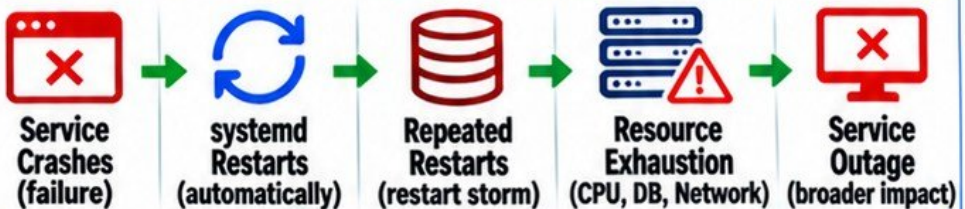
The Restart Loop That Turned One Failure Into a Server Outage

Production Incident: A service repeatedly crashes, restarts, and exhausts shared infrastructure resources.

Key Learnings

1. Examine Restart=, RestartSec=, StartLimitIntervalSec=, and StartLimitBurst=.
2. Understand how restart storms amplify database and network pressure.
3. Investigate systemd unit dependencies and failure propagation.
4. Compare automatic recovery with controlled failure containment.
5. Design startup backoff, dependency checks, and restart-rate alerts.

How a Restart Loop Escalates an Outage



Key systemd Restart Controls

Setting	Purpose
Restart=	When to restart (e.g., no, on-failure, always).
RestartSec=	Wait time before restarting (e.g., 5s, 30s, 1min).
StartLimitIntervalSec=	Time window to limit restart attempts (e.g., 300s = 5 minutes).
StartLimitBurst=	Maximum restart attempts within the interval (e.g., 3, 5, 10).

Restart Strategy Comparison

Approach	Use When
Automatic Restart	The failure is temporary (e.g., brief network issue).
Controlled Restart	The service needs a delay, dependency check, or backoff.
No Restart	The failure is persistent and requires manual investigation (e.g., configuration error).

Practical Safeguards

- Add dependency checks (After=, Requires=) to ensure required services are ready.
- Use a sensible backoff (RestartSec=) to reduce pressure.
- Set restart limits (StartLimitIntervalSec=, StartLimitBurst=) to stop restart storms.
- Monitor restart rate and alert on abnormal patterns (e.g., > 3 restarts in 5 minutes).

Senior Engineer Decision: Automatic restart is a recovery mechanism only when the failure is recoverable.

Linux File Descriptors: The Limit Nobody Notices Until Production Breaks

Production Incident: Applications reject connections even though CPU, memory, and disk capacity look healthy.

Key Learnings

1. Understand per-process descriptor limits and system-wide file table capacity.
2. Examine sockets, pipes, files, and leaked descriptors.
3. Investigate /proc/<pid>/fd, /proc/<pid>/limits, and file-nr.
4. Distinguish descriptor exhaustion from TCP connection and backlog limits.
5. Compare systemd LimitNOFILE= with shell-level ulimit.

What Are File Descriptors?

A file descriptor (FD) is a number the kernel uses to track open resources for a process, such as:

- Files (logs, data, configs)
- Sockets (network connections)
- Pipes (process communication)
- Devices (disk, network, etc.)
- Other kernel resources

Useful Investigation Commands

Command	What It Shows
ls /proc/<pid>/fd	Lists all open file descriptors for a process.
cat /proc/<pid>/limits	Shows per-process limits (e.g., open files).
cat /proc/sys/fs/file-nr	System-wide file table usage (allocated, used, max).
ulimit -n	Current shell limit for open files.
systemctl show <svc> | grep LimitNOFILE	File descriptor limit set for a systemd service.

Key Limits and Their Scope

Limit	Scope and Meaning
ulimit -n	Per-user (shell) limit for open files (e.g., 1024, 4096).
LimitNOFILE=	Systemd service limit for open files (overrides shell ulimit).
file-nr	System-wide file table usage (allocated, used, maximum).
somaxconn	TCP listen backlog (pending connections queue).

Descriptor Exhaustion vs Network Limits

Descriptor Exhaustion (FD Limit Reached)

- The process cannot open more files, sockets, or connections.
- Common error: Too many open files.
- Caused by leaks, high connection count, or a low limit.

TCP Connection Limits (Backlog/Networking)

- New connections are dropped or refused.
- Controlled by listen backlog (e.g., somaxconn) and network settings.
- Not the same as file descriptor exhaustion, but often confused with it.

Architecture Trade-Off: Raising limits increases capacity but does not fix descriptor leaks.

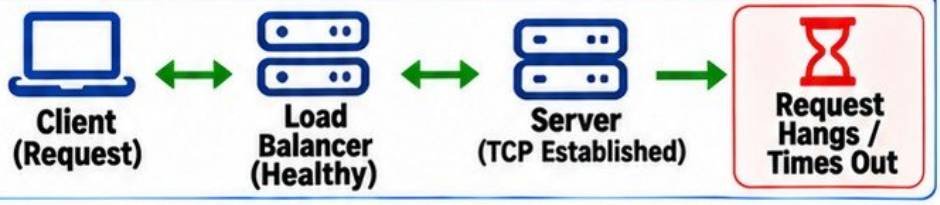
TCP Connections Are Established, but Requests Never Complete

Production Incident: A load balancer reports healthy targets, yet clients experience intermittent timeouts.

Key Learnings

1. Examine TCP send and receive buffers, retransmissions, and congestion control.
2. Understand connection states and application-level request processing.
3. Investigate ss -ti, nstat, and tcpdump.
4. Distinguish packet loss from application queueing and downstream latency.
5. Identify when increasing timeouts makes resource exhaustion worse.

What Can Go Wrong?



TCP and Application Flow



Useful Investigation Commands

Command	What It Shows
ss -ti	Active TCP connections with detailed TCP state (RTT, retransmissions, send/recv queues, congestion).
nstat	TCP statistics (retransmits, errors, drops, congestion metrics).
tcpdump	Capture and analyze packets (loss, retransmission, latency).

Key TCP Indicators to Check

Indicator	Meaning
Send-Q / Recv-Q	Growing queues indicate the receiver or application is slow.
Retransmissions	Packets are being resent (often due to loss or congestion).
RTT (Round-Trip Time)	Higher values indicate network or downstream delay.
Connection State	e.g., ESTABLISHED, TIME_WAIT, CLOSE_WAIT (helps spot issues).

Connectivity vs Application Responsiveness

TCP Established Means

- The network path is open.
- The client and server completed the handshake.
- The connection is active and can carry data.

But It Does NOT Mean

- The application is processing requests.
- The request will complete quickly.
- Downstream services (DB, API) are healthy.
- There is no internal queue or resource bottleneck.

Senior Engineer Decision: An established TCP connection proves connectivity, not application responsiveness.

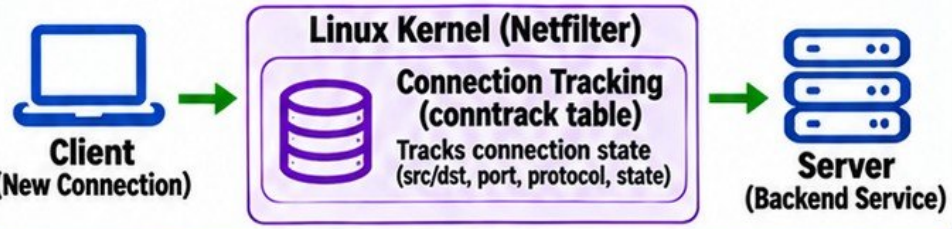
The Linux Connection Tracking Table That Took Down the Network

Production Incident: New connections fail intermittently while established sessions continue working.

Key Learnings

1. Understand Linux Netfilter connection tracking (conntrack).
2. Investigate nf_conntrack_count and nf_conntrack_max.
3. Examine NAT behavior, connection lifetimes, and high-cardinality traffic.
4. Distinguish conntrack exhaustion from application connection-pool limits.
5. Analyze the effect of increasing table capacity on kernel memory consumption.

How Connection Tracking Works



Key conntrack Components

Component	Meaning
nf_conntrack_count	Current number of active connection entries.
nf_conntrack_max	Maximum number of entries the table can hold.
NAT	Rewrites IP/port (e.g., private → public) and creates conntrack entries.
Connection Lifetime	How long entries stay in the table (e.g., ESTABLISHED, TIME_WAIT).
High-Cardinality Traffic	A large number of unique connections (e.g., many clients, short-lived connections, or scans).

Useful Investigation Commands

Command	What It Shows
cat /proc/sys/net/nf_conntrack_count	Current conntrack entry count (usage).
cat /proc/sys/net/nf_conntrack_max	Maximum allowed entries (capacity).
conntrack -L | head	List recent connection tracking entries.
sysctl -a | grep conntrack	All conntrack-related settings (e.g., timeouts, limits).

Conntrack Exhaustion vs App Pool Limits

Conntrack Exhaustion (Kernel Level)

- New connections are dropped or delayed.
- nf_conntrack_count is close to nf_conntrack_max.
- Affects all network traffic (shoulder, not single app).

App Connection-Pool Limit (Application Level)

- Only one application is affected.
- The kernel still accepts new connections.
- Check the application's connection pool settings (e.g., DB, API, pool size).

Production Insight: Connection tracking is a shared kernel resource. One workload can affect unrelated services.

DNS Works in the Terminal but Fails Inside the Application

Production Incident: dig resolves a hostname correctly, but production application requests fail.

Key Learnings

1. Examine glibc name resolution, /etc/nsswitch.conf, and /etc/resolv.conf.
2. Understand systemd-resolved, DNS caching, search domains, and resolver timeouts.
3. Investigate application-specific resolver behavior.
4. Compare DNS lookup success with application connection success.
5. Diagnose using resolvectl, getent hosts, dig, and packet captures.

The DNS Resolution Path in Linux



Key DNS Components and Roles

Component	Role
glibc resolver	Library used by most Linux applications for DNS lookups.
/etc/nsswitch.conf	Controls the order and sources for name resolution (e.g., files, dns, mdns).
/etc/resolv.conf	Lists DNS servers, search domains, and timeout options (managed directly or by systemd-resolved).
systemd-resolved	Local DNS resolver and cache that handles requests from apps.
Search domains	Automatically appends domains (e.g., service.local) during lookup.

Useful Investigation Commands

Command	What It Shows
dig <hostname>	Query DNS directly and view the response.
resolvectl status	DNS configuration and current resolver settings.
getent hosts <name>	Resolve a hostname using the system resolver (glibc path).
cat /etc/nsswitch.conf	Name resolution order (e.g., files dns).
cat /etc/resolv.conf	DNS servers, search domains, and options.
tcpdump -n port 53	Capture and inspect DNS queries and responses.

Why the Application Can Fail

- Different resolver path (e.g., container, chroot, or custom DNS config).
- Caching or stale DNS records (systemd-resolved).
- Search domain issues (wrong or missing domain).
- Resolver timeouts or slow response from DNS server.
- Network policy or firewall blocking DNS for the app (e.g., container or service network).

Senior Engineer Decision: Test the resolver path used by the failing application, not merely a convenient DNS command.

The Storage Is Fast, but the Database Is Waiting

Production Incident: A database becomes slow despite apparently acceptable disk throughput.

Key Learnings

1. Distinguish IOPS, throughput, latency, queue depth, and I/O wait.
2. Examine Linux block-layer behavior and storage-device saturation.
3. Investigate iostat -xz, pidstat -d, and /proc/pressure/io.
4. Understand why high throughput does not guarantee low latency.
5. Compare faster storage with query tuning and workload isolation.

Key Storage and I/O Concepts



IOPS (Number of I/O operations per second)



Throughput (Total data transferred per second, e.g., MB/s)



Latency (Time to complete a single I/O)



Queue Depth (Number of pending I/O requests)



I/O Wait (Time the CPU is idle waiting for I/O to finish)

Useful Investigation Commands

Command	What It Shows
iostat -xz 1	Detailed disk I/O metrics (IOPS, throughput, latency, queue depth, %util).
pidstat -d 1	Per-process disk I/O (read/write rate and wait time).
cat /proc/pressure/io	I/O pressure and time spent stalled on disk I/O.
lsblk -o NAME,ROTA,SIZE	Block devices and key characteristics (e.g., HDD vs SSD).
nvme top (or nvme-cli)	NVMe device performance and utilization (if applicable).

Why a Fast Disk Can Still Be Slow

Scenario	Explanation
High queue depth	Too many I/O requests waiting in the device queue.
Storage saturation	The device is busy (high %util) even with good throughput.
Small random I/O	Many small reads/writes can increase latency.
Database I/O pattern	Inefficient queries (e.g., full table scans) can create heavy I/O.
I/O contention	Multiple workloads competing for the same storage.

Diagnose and Respond



Check I/O metrics (iostat -xz) for high latency, queue depth, or device saturation.



Identify the database process I/O usage (pidstat -d).



Review database queries and indexes for inefficient I/O patterns.



Consider workload isolation (e.g., separate volume or dedicated storage) if needed.



Evaluate storage upgrade only after confirming I/O is the bottleneck.



Architecture Trade-Off: Storage upgrades may hide inefficient I/O patterns without resolving contention.

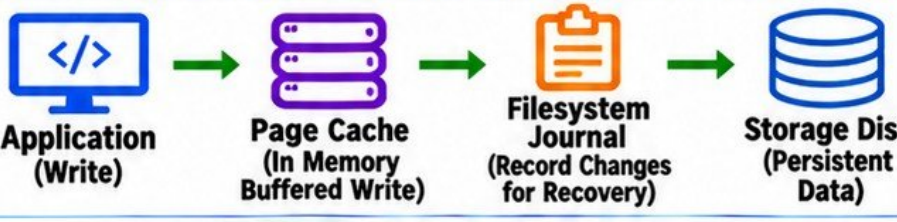
The Filesystem Crash That Exposed a False Durability Assumption

Production Incident: An application acknowledges a successful write, but recent data disappears after a crash.

Key Learnings

1. Understand page cache, buffered writes, fsync, and durability boundaries.
2. Examine filesystem journaling and crash consistency.
3. Distinguish atomic file replacement from durable persistence.
4. Investigate write ordering and application acknowledgment behavior.
5. Compare durability guarantees with performance costs.

The Write Path and Where Data Lives



Key Concepts and Definitions

Concept	Meaning
Page Cache	In-memory buffer that holds writes before they reach disk.
Buffered Write	A write considered successfiul by the OS, but not yet durable.
fsync	Forces data and metadata from the page cache to disk.
Journaling	Records filesystem changes so the system can recover consistently after a crash.
Crash Consistency	The filesystem returns to a valid, consistent state after failure.
Atomic Replacement	Write to a temporary file and rename (e.g., tmp → final) so the change is all-or-nothing.
Durability	A guarantee that data will survive a crash or power loss.

Useful Investigation Commands

Command	What It Shows
sync	Flushes pending writes from memory to disk.
strace -e fsync <app>	Verifies whether the application calls fsync.
dmesg | grep -i ext4	Filesystem and journaling messages (e.g., errors, recovery).
mount | grep <fs>	Filesystem type and mount options (e.g., ext4, xfs, commit).
journalctl -k	Kernel logs for possible I/O or filesystem issues.

Durability vs Performance Trade-Off

Stronger Durability (Safer)

- Use fsync or fdatasync.
- Rely on journaled filesystems (e.g., ext4, xfs).
- Apply atomic file replacement (write + rename).
- Lower risk of data loss after a crash.

Higher Performance (Riskier)

- Rely only on buffered writes (page cache).
- Lower write latency and higher throughput.
- Data can be lost if a crash happens before the write reaches disk.

Senior Engineer Decision: Define precisely when the application may truthfully acknowledge that data is durable.

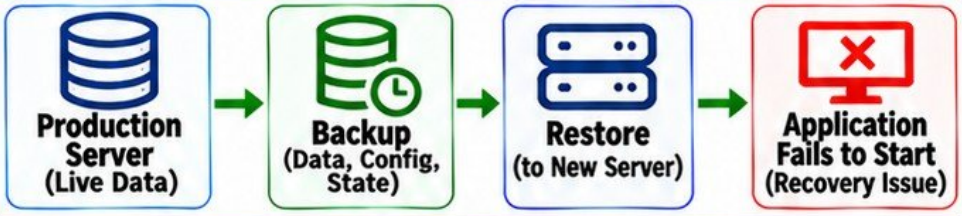
The Backup Completed Successfully. The Recovery Failed.

Production Incident: A Linux server is rebuilt from backup, but applications cannot start.

Key Learnings

1. Examine permissions, ownership, ACLs, extended attributes, and SELinux labels.
2. Investigate missing systemd units, mount definitions, secrets, and dependencies.
3. Compare file backups with application-consistent recovery.
4. Validate restore operations in isolated environments.
5. Measure recovery time and verify application data integrity.

Backup vs Recovery: The Complete Picture



Common Recovery Failure Causes

Area	Examples of What Can Be Missing or Wrong
Permissions & Ownership	Incorrect user, group, or file permissions.
ACLs & Extended Attributes	Missing ACLs or important file attributes (e.g., capabilities, xattrs).
SELinux Labels	Wrong or missing security contexts (e.g., httpd_sys_content_t).
System Configuration	Missing systemd units, mount entries (/etc/fstab), secrets, or environment files.
Dependencies	Required services, packages, or network resources not available.

Useful Investigation Commands

Command	What It Shows
ls -l /path	File permissions and ownership.
getfacl <file>	ACLs on files and directories.
lsattr <file>	Extended attributes (e.g., immutable).
ls -Z <file>	SELinux security context (label).
systemctl list-unit-files	Available and missing systemd units.
cat /etc/fstab	Filesystem mount definitions.
restorecon -Rv /path	Restore correct SELinux labels.

File Backup vs Application Recovery

File Backup (Input, Not Complete)

- Copies files and data.
- May miss permissions, ACLs, and SELinux labels.
- May not include systemd units, mounts, secrets, or runtime state.
- Backup can succeed even if the application cannot be restored.

Application Recovery (The Real Outcome)

- Restores data, configuration, and security context correctly.
- Recreates required services, mounts, and dependencies.
- Verifies the application actually starts and works.
- Includes validation and integrity checks.

Production Insight: Backup success is an input metric. Successful restoration is the reliability outcome.

Root Inside a Container Is Not the Same as Root on the Host

Production Incident: A compromised container attempts to access host resources.

Key Learnings

1. Examine Linux namespaces, cgroups, capabilities, and seccomp.
2. Understand user namespaces and UID mapping.
3. Investigate privileged containers, host mounts, and container runtime sockets.
4. Compare rootless execution with conventional container isolation.
5. Evaluate how kernel vulnerabilities can cross container boundaries.

Container Isolation: Key Building Blocks



Container Security: Common Risks and Controls

Risk / Misconfiguration	What It Means
Privileged Container	Gives almost full access to the host (not isolated).
Host Mounts (e.g., /)	Exposes host filesystem to the container.
Container Runtime Socket (e.g., /var/run/docker.sock)	The container can control the container runtime (escape to host).
Excess Capabilities (e.g., NET_ADMIN, SYS_ADMIN)	Grants powerful host-level operations.
Kernel Vulnerability	A kernel flaw can be exploited from the container to affect the host.

Useful Investigation Commands

Command	What It Shows
`docker inspect <ctr>`	Container configuration (privileged, mounts, capabilities).
`lsns -t all`	Linux namespaces and isolation (PID, NET, MNT, etc.).
`cat /proc/<pid>/status`	Capabilities and user/UID mapping.
`grep CapEff /proc/<pid>/status`	Effective capabilities (e.g., SYS_ADMIN).
`mount | grep <ctr>`	Host mounts visible to the container.

Rootless vs Conventional Containers

Rootless Container (Safer)

- Runs as a non-root user on the host.
- Uses user namespaces (UID mapping).
- Reduced risk of host access and privilege escalation.
- Suitable for most applications.

Conventional Container (Higher Risk)

- Typically runs with root inside the container.
- Can be given privileged mode or host mounts.
- Greater risk if misconfigured or the kernel is vulnerable.
- Use only when necessary and with strict controls.

Security Decision: Container isolation is not equivalent to a separate kernel security boundary.

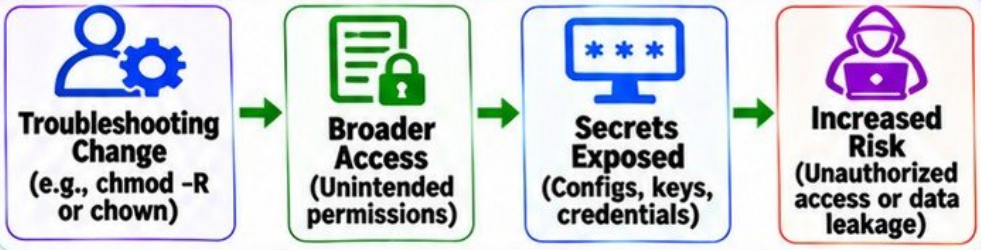
The Permission Change That Accidentally Exposed Production Secrets

Production Incident: A troubleshooting change grants unintended access to sensitive configuration.

Key Learnings

1. Examine Unix permissions, ACLs, capabilities, and service identities.
2. Understand the risks of recursive permission changes (e.g., chmod -R).
3. Investigate secret exposure through process arguments, environment variables, and /proc.
4. Compare DAC permissions with SELinux or AppArmor enforcement.
5. Design safe permission fixes and verify access after the change.

How the Exposure Can Happen



Access Controls and Security Layers

Control	What It Does
Unix Permissions (DAC)	File ownership and read, write, execute (rwx) control.
ACLs	Fine-grained access for specific users or groups.
Capabilities	Limited, specific privileges (e.g., CAP_NET_BIND_SERVICE).
SELinux	Mandatory access control with policy enforcement.
AppArmor	Application-level access control using security profiles.
Service Identities	Dedicated users/groups for running services with least privilege.

Useful Investigation Commands

Command	What It Shows
ls -l /path	File permissions, owner, and group.
getfacl /path	ACL entries for a file or directory.
ps -ef | grep <app>	Running process and its arguments (check for exposed secrets).
cat /proc/<pid>/environ	Environment variables for a process (look for credentials).
ls -Z /path	SELinux context of a file or directory.
aa-status	AppArmor status and enforced profiles.

Safe Remediation and Verification

- Apply the smallest necessary permission change (e.g., chown <user>:<group> <file>).
- Avoid broad changes like chmod -R 777.
- Review file, process, and environment access (permissions, ACLs, /proc).
- Verify with SELinux/AppArmor policies if in use.
- Retest the application and confirm no unintended access was introduced.

Senior Engineer Decision: Fix the smallest permission boundary necessary. Never make broad access changes merely to silence an error.

The Linux Host Was Compromised, but the Logs Look Normal

Production Incident: Suspicious outbound traffic appears without corresponding application logs.

Key Learnings

1. Distinguish application logging from kernel, audit, and network evidence.
2. Examine auditd, journald, process ancestry, and execution history.
3. Investigate suspicious systemd services, timers, and persistence mechanisms.
4. Understand why host-local logs may be untrustworthy after compromise.
5. Design remote log forwarding, integrity controls, and incident containment.

Evidence Beyond Application Logs

Application Logs (App activity, (imited scope) → **Kernel Logs** (journald) (System events, services, errors) → **Audit Logs** (auditd) (Sensitive actions, exec, config changes) → **Network Evidence** (Connections, outbound traffic, DNS, etc.)

Key Investigation Areas

Area	What to Check
Process Ancestry	Parent/child processes (pstree, ps -ef).
Systemd Services	Unknown or newly added services (systemctl list-units).
Timers	Suspicious scheduled tasks (systemctl list-timers).
Execution History	Command history and recent executions (e.g., ~/.bash_history, auditd).
Outbound Activity	Unexpected connections (ss, netstat, tcpdump).
Persistence	Mechanisms to survive reboot (services, timers, cron, RC scripts, etc.).

Useful Investigation Commands

Command	What It Shows
journalctl -xe	System and service logs (detailed).
ausearch -i	Audit events in readable format (e.g., exec, file access, changes).
ps -ef --forest	Process tree and parent/child relationships.
systemctl list-units	All loaded systemd services (active and inactive).
systemctl list-timers	Configured timers and their next run time.
ss -tupn	Active network connections with processes.

Detection, Protection and Response

- Correlate application, system, audit, and network logs to build the full picture.
- Forward logs to a remote, trusted log store (e.g., SIEM, log server, cloud logging).
- Use log integrity controls (e.g., immutability, signing, or append-only storage).
- Contain the incident (isolate the host, preserve evidence, and analyze offline).
- Rebuild from a known-good image if compromise is confirmed.

Security Insight: A compromised host should not be treated as the sole source of truth about its own activity.

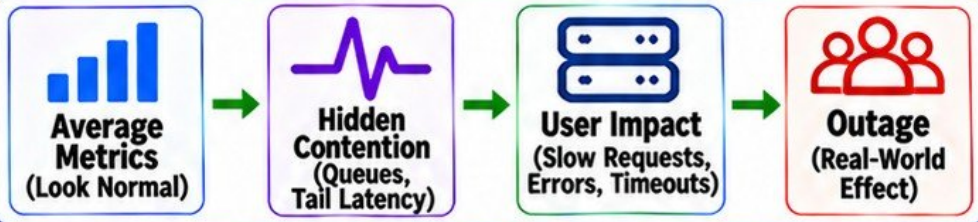
The Metrics Look Normal Because You Are Measuring the Wrong Thing

Production Incident: Infrastructure dashboards show healthy averages during a severe user-facing outage.

Key Learnings

1. Examine the limitations of average CPU, memory, and network metrics.
2. Understand tail latency, saturation, and pressure stall information.
3. Investigate per-process and per-cgroup resource pressure.
4. Correlate application errors with kernel and infrastructure signals.
5. Design alerts that distinguish symptoms from likely causes.

Averages vs Real User Impact



Key Metrics: What to Measure and Why

Metric / Signal	What It Tells You
Tail Latency (p95/p99)	User experience at the slowest end (where pain shows).
Saturation	Whether a resource is at or near its limit.
Pressure Stall Information (PSI)	Time tasks are stalled due to CPU, memory, or I/O pressure.
Per-Process Metrics	Which process is actually consuming or blocking resources.
Per-cgroup Metrics	Resource usage and pressure for a specific workload (e.g., a container).
Error Rate	Application health and user-facing failures.

Useful Investigation Commands

Command	What It Shows
top / htop	Real-time CPU and memory usage by process.
pidstat -u -r -d	Per-process CPU, memory, and I/O activity.
cat /proc/pressure/*	CPU, memory, and I/O pressure (PSI).
iostat -xz	Disk I/O, latency, and queue depth.
ss -s	Socket and connection summary (possible saturation).
journalctl -u <service>	Application errors and failure patterns.

Better Alert Design

Effective Alerts (Actionable)

- Tail latency above threshold (e.g., p99 > 2s).
- Resource saturation (e.g., high run queue, I/O wait, PSI pressure).
- Rising error rate (e.g., HTTP 5xx, timeouts).
- User impact signals (e.g., failed requests, slow transactions).

Ineffective Alerts (Noisy or Misleading)

- Average CPU is at 40% (looks healthy but can hide contention).
- Average memory usage is normal.
- Network usage is below threshold.
- Single-metric alerts without context (e.g., CPU only).

Senior Engineer Decision: Measure resource contention and user impact, not just resource consumption.

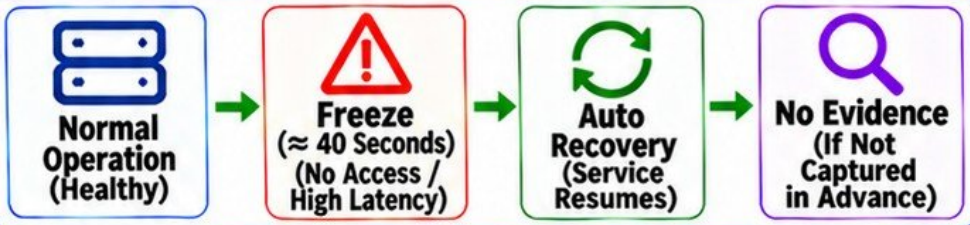
The Production Incident That Disappeared Before Anyone Investigated

Production Incident: A server freezes for 40 seconds and recovers before engineers can collect evidence.

Key Learnings

1. Examine intermittent CPU stalls, I/O hangs, memory reclaim, and kernel warnings.
2. Use persistent journaling, historical metrics, and system activity collection.
3. Understand when eBPF-based tracing can reveal short-lived failures.
4. Design low-overhead diagnostic capture before the next incident.
5. Evaluate tracing overhead, privacy, and operational risk.

A Transient Incident Timeline (Example)



Possible Short-Lived Causes

Area	Examples of What to Investigate
CPU	Scheduling stalls, high load, or soft lockups.
I/O	Disk I/O hangs, high queue depth, storage latency.
Memory	Memory reclaim, swapping, or pressure (OOM risk).
Kernel	Kernel warnings, hung tasks, or hardware errors (e.g., dmesg).
System / Services	Service timeouts, dependency stalls, or resource contention.

Useful Investigation Commands

Command	What It Shows
journalctl -k	Kernel messages and warnings.
journalctl -u <svc>	Service logs during the time window.
sar -u -r -d	Historical CPU, memory, and disk metrics.
dmesg -T	Kernel events with human-readable time.
bpftool prog list	View loaded eBPF programs.
perf record/report	Capture short-lived CPU activity for analysis.

Build Preparedness for the Next Incident

- Enable persistent journald with log retention.
- Collect and store time-series metrics (e.g., Prometheus).
- Use eBPF tracing for low-overhead, on-demand visibility (e.g., bpftrace).
- Set up always-on or ring-buffer diagnostics (e.g., sysdig, perf, or eBPF).
- Balance visibility with privacy and operational risk (consider data sensitivity and overhead).

Production Insight: Intermittent failures require evidence collection designed before the failure happens.

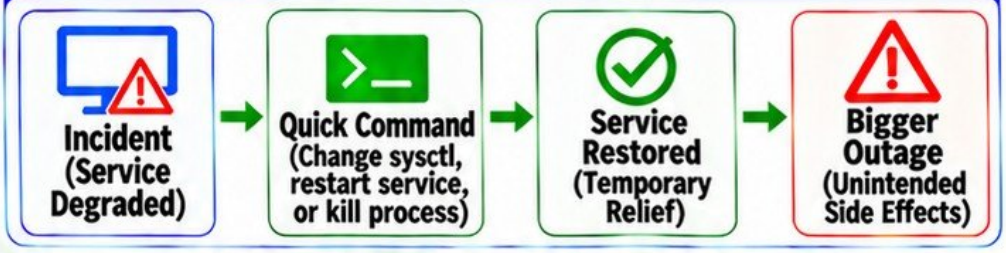
The Linux Command That Fixed the Incident but Created a Bigger One

Production Incident: A service is restored after a quick Linux change, but a second, larger outage follows.

Key Learnings

1. Examine the risks of sysctl changes, process termination, and service restarts.
2. Understand how local remediation can affect shared kernel resources.
3. Compare reversible mitigation with destructive intervention.
4. Use the observe, confirm, change, verify model.
5. Define rollback conditions, blast-radius limits, and incident authority.

A Realistic Incident Flow



High-Risk Remediation Actions

Action	What It Does	Typical Risks
sysctl Change	Modify kernel behavior (e.g., networking, memory).	System-wide impact, performance regression, or instability.
Service Restart	Stop and start a service (e.g., systemctl restart ...)	Disrupts dependent services or active connections.
Kill Process	Forcefully terminate a process (e.g., kill -9 <pid>).	Data loss, partial state, or cascade failures.
Networking Restart	Reset networking (e.g., systemctl restart NetworkManager).	Connection loss, routing disruption, or service isolation.

The Observe, Confirm, Change, Verify Model

Step	What to Do
Observe	Collect evidence (metrics, logs, affected scope, and symptoms).
Confirm	Validate the root cause and likely impact of the change.
Change	Apply the smallest, safest mitigation (with a rollback plan).
Verify	Confirm service recovery and check for side effects (other services, users, and metrics).

Safe Change Checklist

- [x] Assess the change scope and blast radius.
- [x] Use the least disruptive, reversible option.
- [x] Have a tested rollback command or plan.
- [x] Communicate and get incident authority (as required).
- [x] Verify not only the fix, but also system-wide impact after the change.

Senior Engineer Decision: The safest mitigation is not necessarily the fastest command. It is the change with the best understood consequences.

Final Linux Black Book Incident:
Five Symptoms, One Hidden Failure Chain

Production Scenario:

A production Linux host runs an API service, an Nginx reverse proxy, and containerized background workers.

Observed Symptoms:

- Intermittent 502 errors for users.
- CPU utilization is moderate.
- Available memory appears healthy.
- Disk capacity is sufficient.
- Service manager reports all processes running.
- New outbound connections intermittently fail.
- Worker queues grow and API latency increases.



Investigation Objectives:

1. Determine the failure domain before restarting services.
2. Correlate TCP retransmissions, socket states, connection tracking, and queueing.
3. Inspect cgroup pressure, file descriptor usage, and process scheduling delays.
4. Identify whether the incident is one bottleneck or interacting failures.
5. Choose a mitigation that avoids worsening traffic pressure or destroying evidence.
6. Verify recovery using latency, error rates, queue depth, and resource pressure.
7. Produce a root-cause analysis, corrective actions, and prevention plan.

Investigation Workflow

Define Failure Domain → Collect Evidence (Network, System, App) → Correlate Signals (TCP, Queues, Pressure) → Confirm Root Cause (Single or Interacting) → Mitigate Safely (Low Risk, Reversible) → Verify Recovery (Latency, Errors, Queues)

Key Linux Checks (Examples)

Command	What It Tells You
ss -s	Socket usage and states (ESTAB, TIME-WAIT, etc.).
ss -tan state established	Active connections and volume.
nstat -az	TCP retransmissions and errors.
cat /proc/sys/net/netfilter/nf_conntrack_*	Connection tracking usage and limits.
systemd-cgtop	Per-cgroup CPU, memory, and I/O pressure.
lsof -n | wc -l	Total open file descriptors (may indicate leaks).
pidstat -w -p <pid>	Process scheduling delays (run queue and waits).

Root-Cause & Action Plan

Root Cause Analysis
- State the primary cause.
- Identify contributing factors (if any).

Corrective Actions
- Fix the immediate issue safely.
- Restore normal service levels.

Prevention Plan
- Add monitoring and alerts (e.g., connections, queues, fd, pressure).
- Set resource limits and tuning (e.g., conntrack, ulimits, workers).
- Document and run a follow-up review.

Final Outcome: A verified, stable recovery with a clear root cause, documented corrective actions, and a prevention plan to avoid recurrence.